

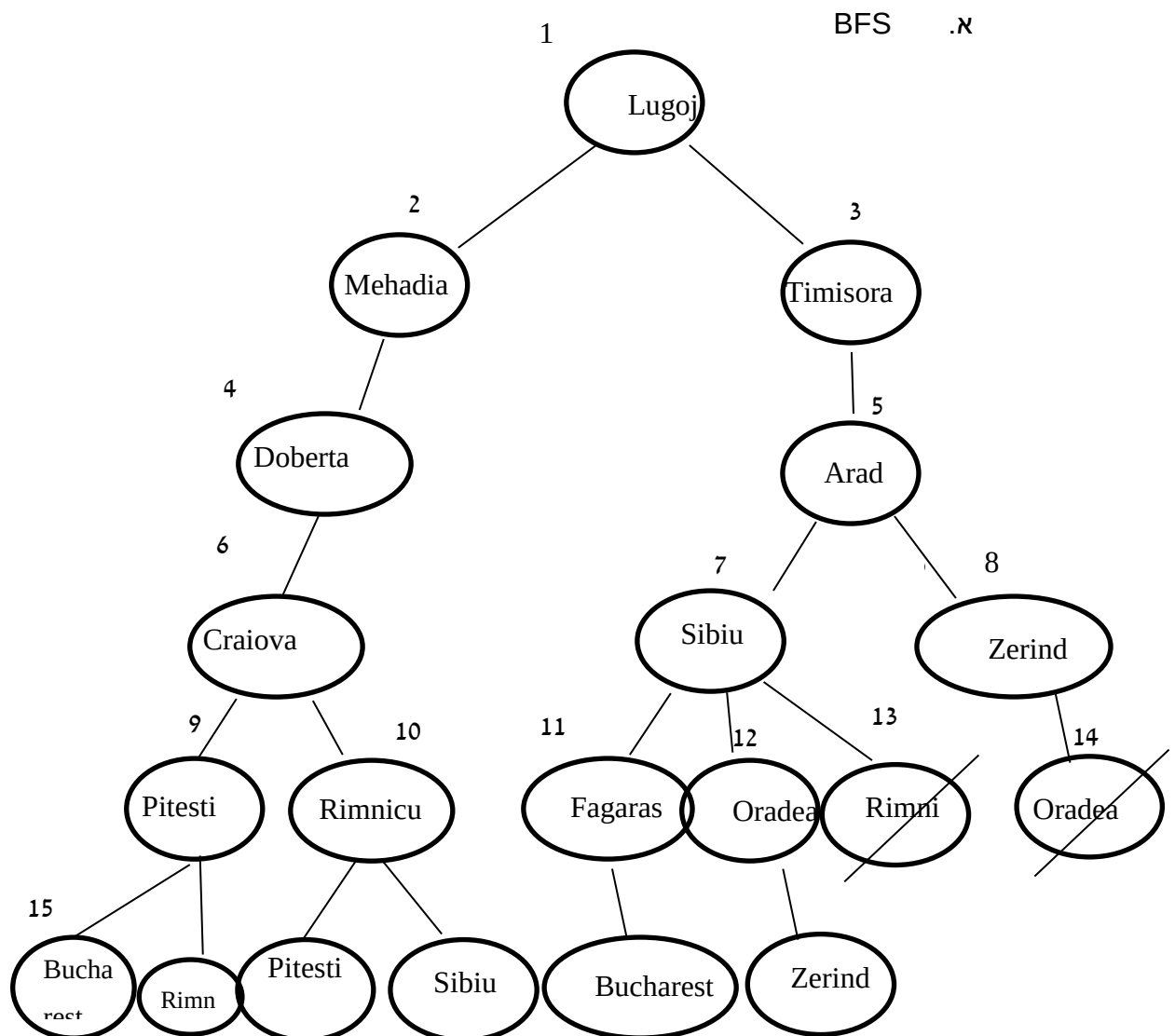
בינה מלאכותית תרגיל מספר 2

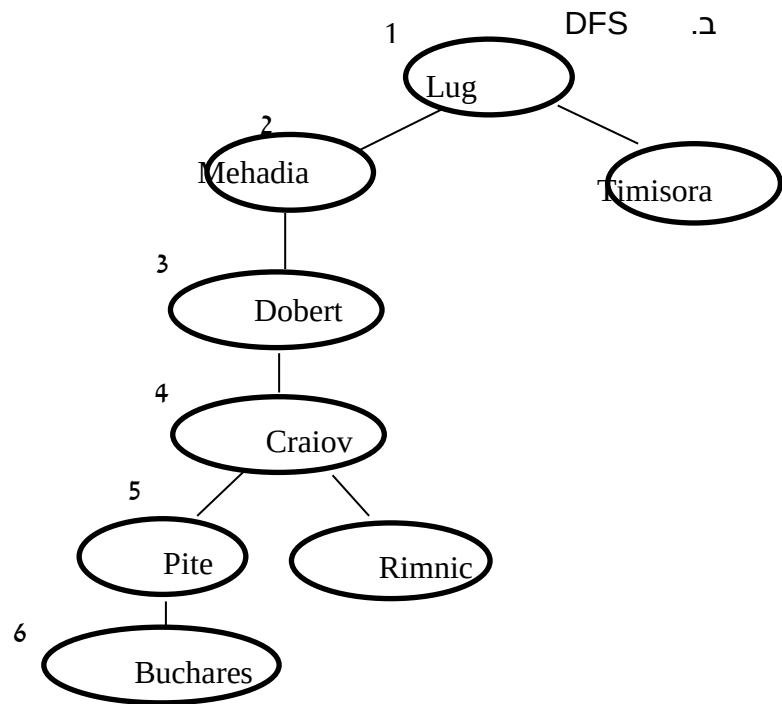
1. [60 נקודות] נחפש במפת רומניה את המסלול הקצר ביותר מ**Lugoj לבוקרסט**.

עבור כל אחד מהאלגוריתמים הבאים הראו את סדר פיתוח הקודקודים – ולא הכנסתם

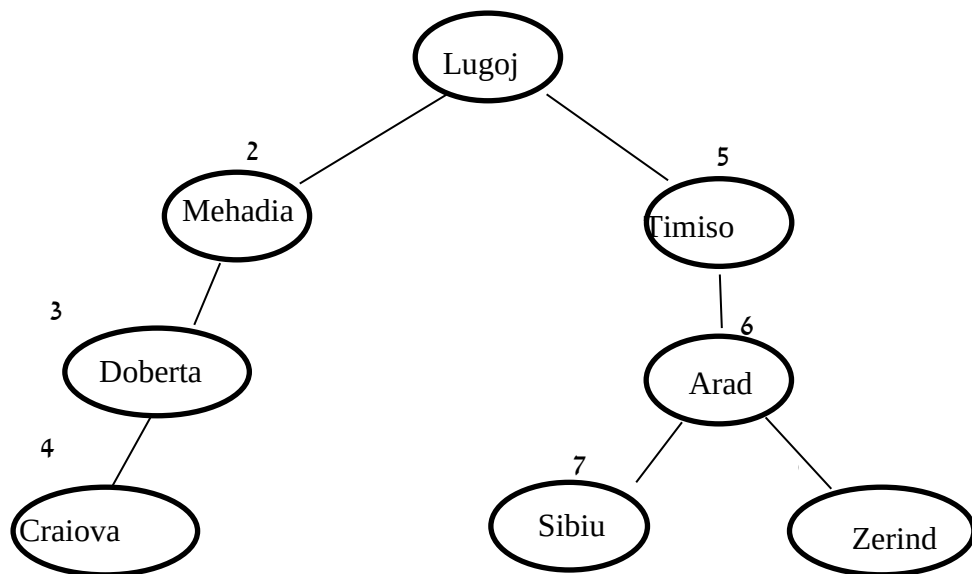
לקבוצת העלים. הניחו שאם שני קודקודים בעלי אותה עדיפות, יפותח קודם הקודקוד

בעל ערך לקסיקוגרפי הנמוך יותר (לפי סדר הא"ב).

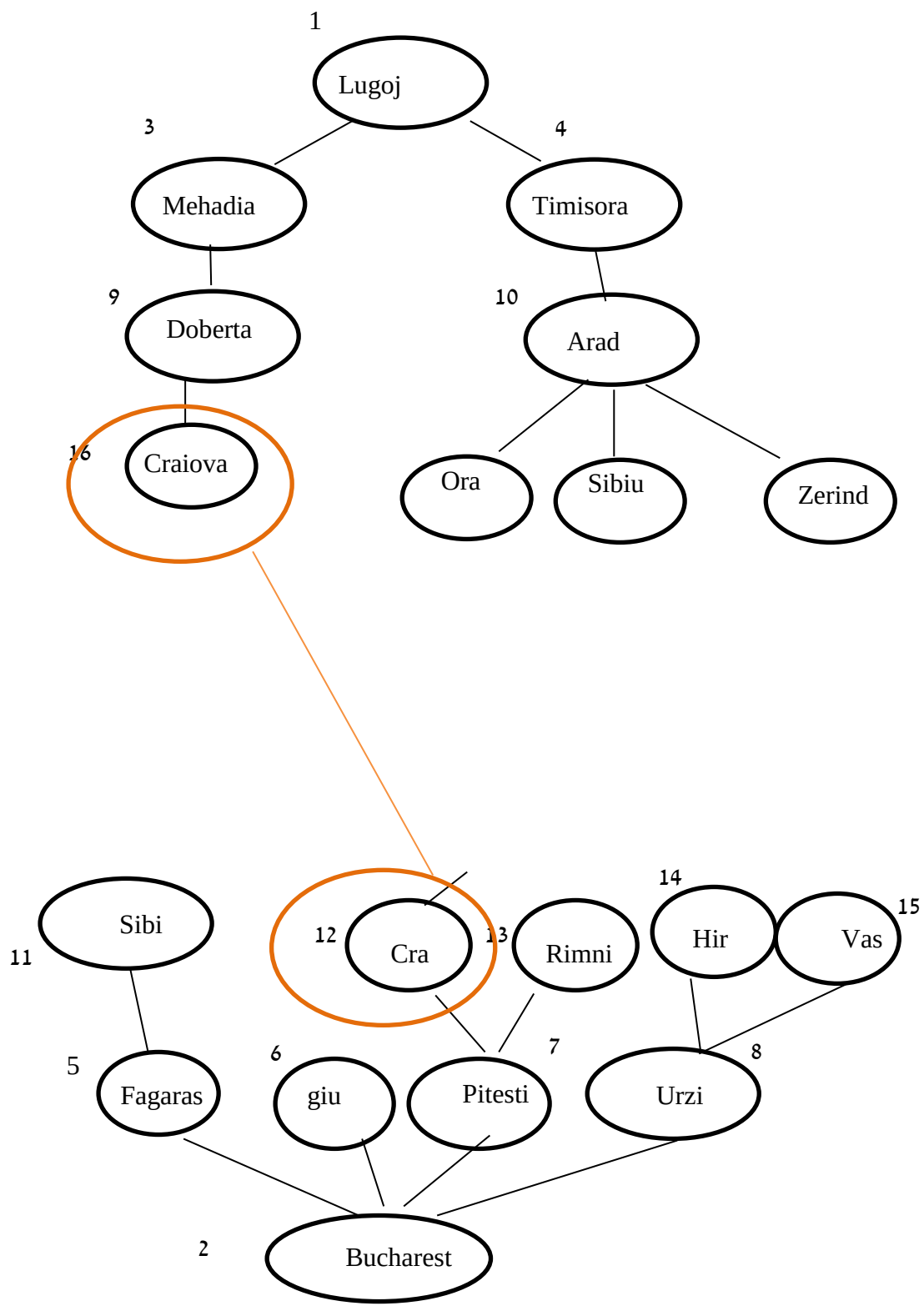




.7 1 Limit =3 Depth limited search

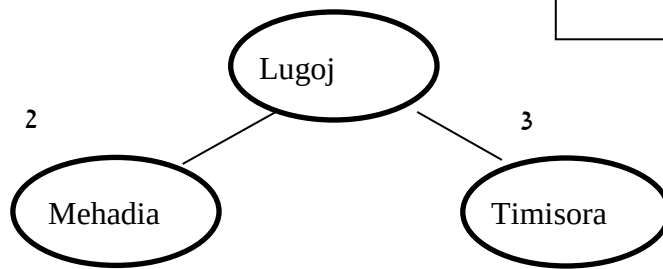


bidirectional Search

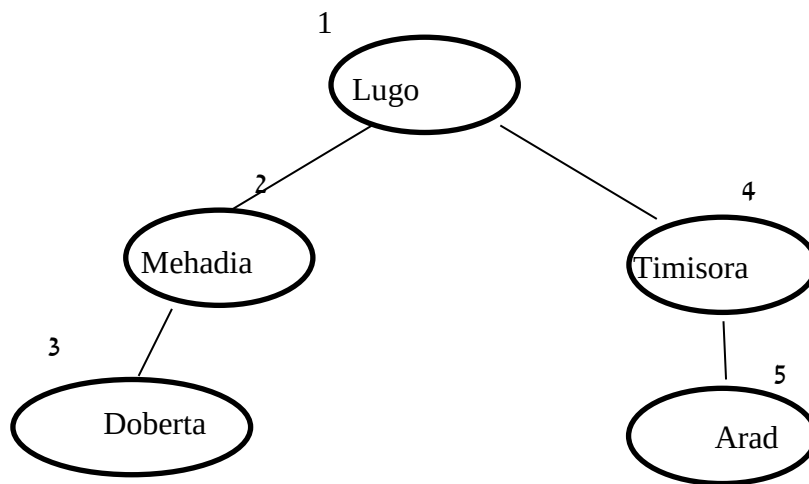


Limit =1

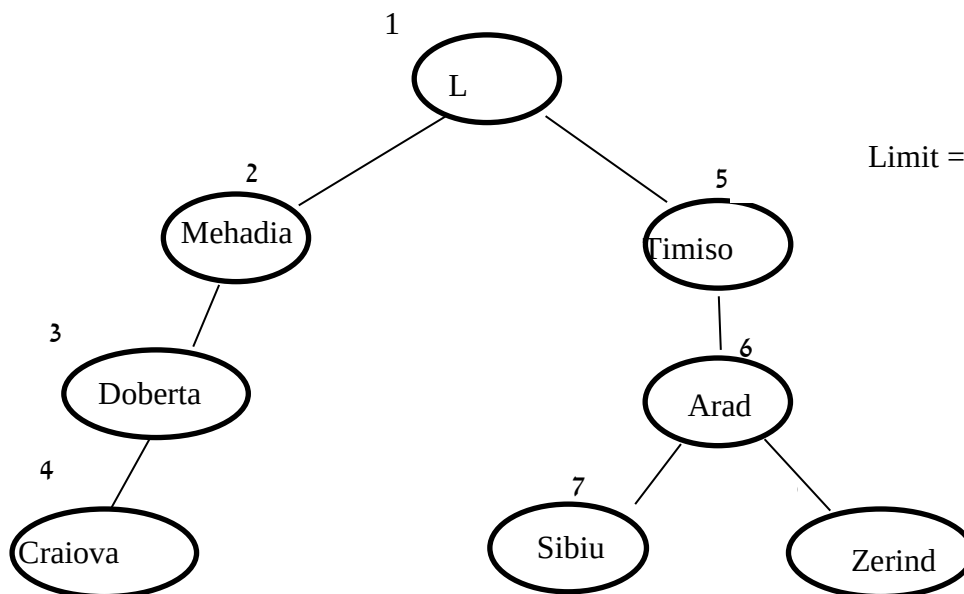
Iterative deepening

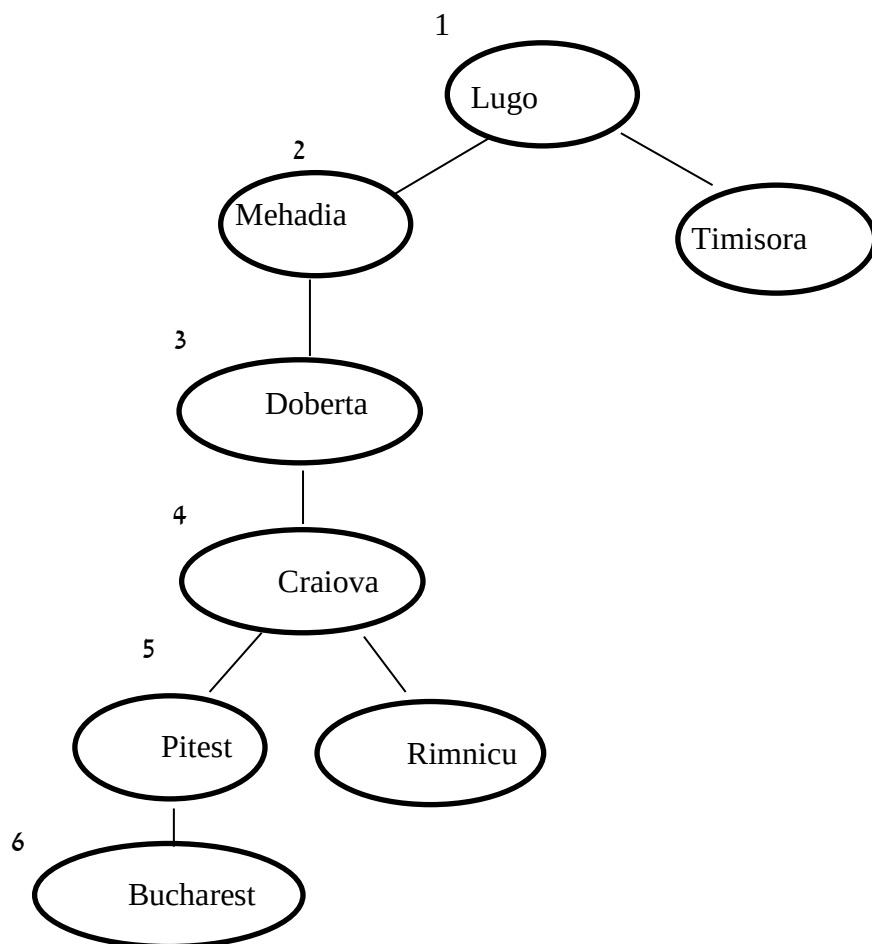
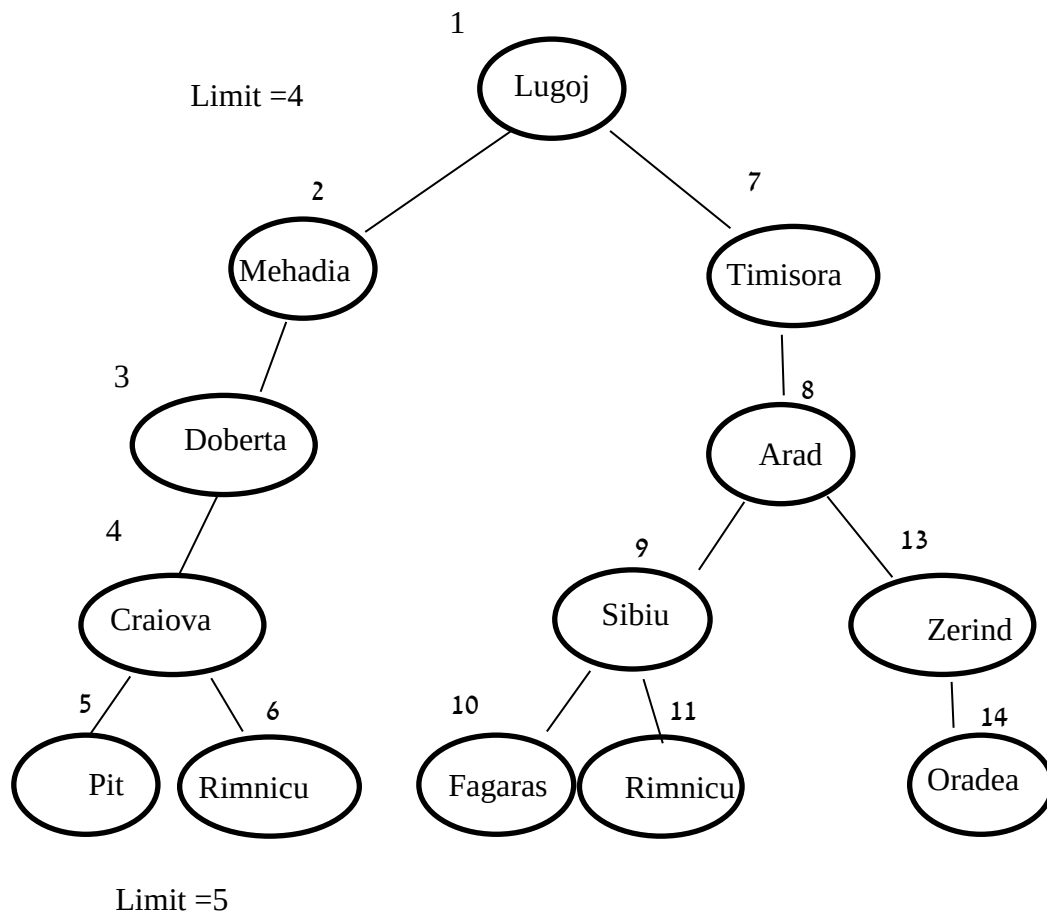


Limit =2



Limit =3





עבור כל משפט, ציינו אם הוא נכון או לא נכון. חובה לנמק בקצרה.

א. הקודקודים שנשמרים ב closed list הם קודקודים עם ערך גרוע שמנסים להימנע מלהגיע אליהם. – לא נכון.
אלו קודקודים שכבר טופלו – נבדקו אם הם המטרה והופעלה עליהם פונקציית העוקב וכל דרכי ההמשך ממצב זה הוכנסו לעץ ולכן לא נרצה לטפל שנית בקודקודים אלו. אין קשר לערך- עלות מסלול שלהם.

ב. פונקציית העוקב מחשבת לכל מצב, לאיזה מצבים כדאי לעבור כדי להתקדם לפתרון יעיל. – לא נכון
פונקציית העוקב מחשבת לכל מצב לאיזה מצבים אפשר לעבור ממנו. היא אינה יודעת מהו מבחן המטרה ואינה קשורה ליצירת פתרון יעיל אלא רק לאפשרות התקדמות חוקית מהמצב הקיים. (גם אם אינה יעילה)

ג. DFS יכול להחזיר את אותו מסלול אל המטרה ש BFS יחזיר.
נכון. יכול לקרות שהמסלול שהוא הולך בו יגיע לפתרון ש BFS יגיע אליו. אם במקרה ילך על מסלול שמוביל לפתרון במספר מינימלי של צעדים.

ד. בעץ חיפוש, יש לשמור בתוך כל קודקוד את כל המסלול מהמצב ההתחלתי עד אליו, כדי שנוכל לשחזר בסוף את המסלול לפתרון. – לא נכון
בכל קודקוד בעץ שומרים רק את ההורה שלו – המצב שהוביל אליו ואז ניתן לשחזר את המסלול לקודקוד בעזרת ההורה של ההורה ואז ההורה שלו וכו'.

ה. בחלק מאלגוריתמי החיפוש יש צורך להיעזר בטבלת גיבוב. -נכון
בחיפוש bidirectional משתמשים בטבלת גיבוב עבור הקודקודים שפותחו בכל צד של החיפוש.
כמו כן בכל האלגוריתמים שמופעלים על מרחב מצבים שהוא גרף ולא עץ, יש צורך להשתמש בטבלת גיבוב עבור ה closed list לא לטפל מספר פעמים באותו קודקוד.

ו. deepening הוא אלגוריתם המנסה לשלב יתרונות של BFS ויתרונות של DFS. -
נכון

בפועל עובד כמו DFS ולכן יש לו דרישת מקום יעילה, אך בשל הלולאה שבכל הרצה מעלה ב1 את הרמה המותרת, הוא חושף קודקודים בסדר של BFS שנותן פתרון בעל מספר רמות מינימלי.

ז. Iterative deepening יעיל יותר מבחינת זמן ריצה מ-BFS. – לא נכון

הוא מפעיל את depth limited search בהרצות חוזרות, כך שעובר פעמים רבות על תתי עץ קטנים, לעומת BFS שעובר פעם אחת על העץ עד שמגיע לפתרון. בסך הכל סיבוכיות הזמן שלהם שווה – אבל בפרטים מדויקים BFS עובר פעם אחת על כל קודקוד שהוא מפתח.

ח. תשובה של cutoff ב depth limited search זהה לתשובה של כישלון, כיוון

שמשמעה שלא הצלחנו להגיע למטרה.

לא נכון – כישלון משמעו שעברנו על כל מרחב המצבים ולא ניתן למצוא בו את המטרה. תשובה של cutoff משמעה שבמסגרת המצבים שיכולנו להגיע אליהם לפי הגבלת העומק, לא נמצאה המטרה. ייתכן ואם יתאפשר לרדת לרמה גדולה יותר ואז נגיע למצבים נוספים, כן תהיה אפשרות להגיע למטרה.